



SH320 Data Engineering

Spring 2026 Semester

Pandas

4 Data Manipulation: Part 1

Hyun Ahn

School of Computing and Artificial Intelligence

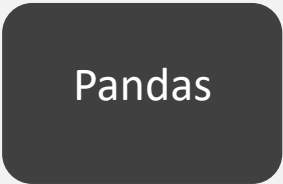
hyunahn@hs.ac.kr



Core Libraries for Data Science

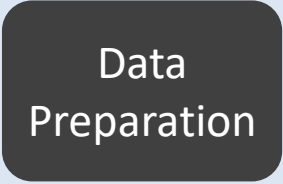
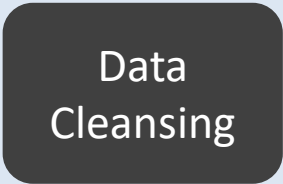
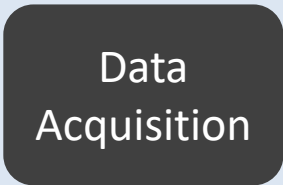


- [W2] N-Dimensional Arrays
 - NumPy arrays
 - Array manipulation
- [W3] Efficient Computation in NumPy
 - Universal functions
 - Advanced features



- [W5] Data Manipulation: Part 1
 - Pandas objects: Series, DataFrame
 - Implicit / explicit indexing

Data Engineering



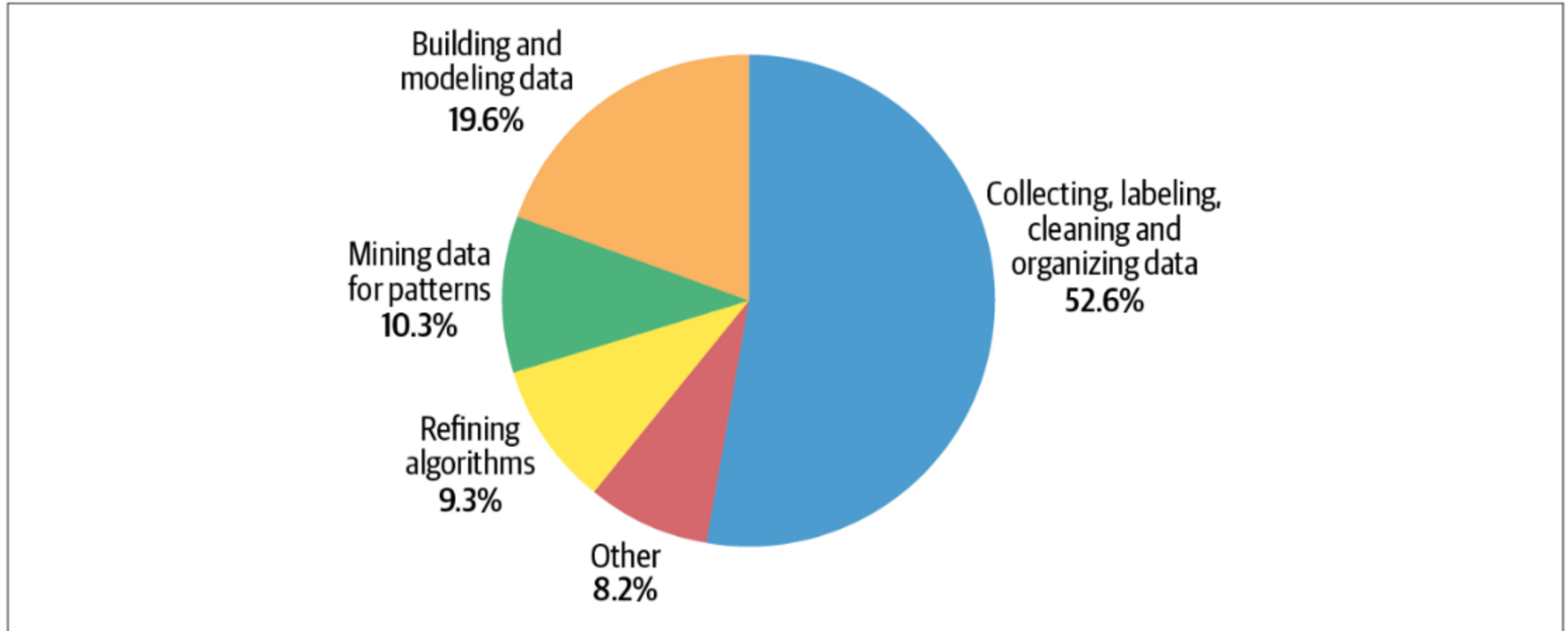
- 데이터 분석·조작에 유용한 파이썬 라이브러리
 - 개발자: 웨스 메키니(Wes McKinney, 2008년)
 - AQR Capital Management 에서 금융 데이터의 계량적 분석을 위해 개발
- 주요 기능
 - 다양한 파일 유형에 대한 **데이터 I/O**(CSV, JSON, HTML, MS Excel, HDF5, ...)
 - 데이터 전처리 & 정제
 - 데이터셋 재구조화/피보팅
 - 레이블 기반 인덱싱/슬라이싱
 - 기술 통계, 시계열 처리

Pandas vs. NumPy



라이브러리		
핵심 객체	ndarray	DataFrame, Series
인덱싱	정수: [0], [1], [2], ...	정수, 레이블(자유롭게 정의 가능)
주요 사용목적	수치계산	데이터 조작·분석
대표적 장점	빠른 다차원 배열 계산	데이터 조작의 편리성

What Tasks are Time-Consuming in D.S.?



Import the Pandas Package

- Pandas 패키지 импорт: `import pandas as pd`
- 헬프 문서 보기: `package_name?`
 - 예: `pd?`, `np?`

```
=====
**pandas** is a Python package providing fast, flexible, and expressive data
structures designed to make working with "relational" or "labeled" data both
easy and intuitive. It aims to be the fundamental high-level building block f
or
doing practical, **real world** data analysis in Python. Additionally, it has
the broader goal of becoming **the most powerful and flexible open source dat
a
analysis / manipulation tool available in any language**. It is already well
on
its way toward this goal.
```

- 버전확인: `package_name.__version__`

```
'0.25.0'
```

Pandas Objects: Series



- Pandas 주요 객체
 - `Series`: 인덱싱된 1차원 배열
 - `DataFrame`: 인덱싱된 2차원 배열($\approx$ 테이블)
 - 리스트/딕셔너리 또는 NumPy 배열로부터 생성 가능

```
ser = pd.Series([0.25, 0.5, 0.75, 1.0])  
  
0      0.25  
1      0.50  
2      0.75  
3      1.00  
dtype: float64
```

Main Attributes



- Pandas 객체의 주요 속성
 - `values`: Pandas 객체의 값들을 NumPy 배열로 반환

```
arr = ser.values  
  
[0.25 0.5  0.75 1.  ]
```

- `index`: 인덱스 값들을 반환

```
ind = ser.index  
  
RangeIndex(start=0, stop=4, step=1)
```


Label-based Indexing



- NumPy(ndarray): 정수형 인덱스(암묵적)
 - [0], [1], [2], ...
- Pandas(Series, DataFrame): 레이블 기반 인덱스(명시적)

```
ser = pd.Series([0.25, 0.5, 0.75, 1.0], index=['a', 'b', 'c', 'd'])
```

```
a    0.25  
b    0.50  
c    0.75  
d    1.00
```

```
ser['b']
```

```
0.5
```

Dictionary and Series



- **Dictionary**: <Key:Value>로 구성되는 파이썬 기본 자료구조
 - 객체 내의 다양한 타입의 Key 정의 가능

```
dict = {'a': 1, 2: 'two', 'third': True}
{'a': 1, 2: 'two', 'third': True}
```

Dictionary and Series



- **Series**: Key 역할을 하는 레이블 기반 인덱스를 사용
 - 가능한 인덱스 유형: 정수, 실수, 문자열, DateTime 등
 - 인덱싱, 슬라이싱을 지원하므로 Dictionary 보다 데이터 조작이 간편

```
ser = pd.Series(dict)
```

⇒ Series 는 1차원 배열인 동시에 Dictionary 와 유사한 객체

Creating a Series Object from a Dictionary



- Dictionary 로부터 Series 객체 생성

```
population_dict = {'California': 38332521,  
                   'Texas': 26448193,  
                   'New York': 19651127,  
                   'Florida': 19552860,  
                   'Illinois': 12882135}
```

```
population = pd.Series(population_dict)
```



```
California    38332521  
Texas         26448193  
New York      19651127  
Florida       19552860  
Illinois      12882135  
dtype: int64
```

- Dictionary 와 달리 Series 는 슬라이싱 같은 배열 연산을 지원

```
population['California':'New York']
```



```
California    38332521  
Texas         26448193  
New York      19651127  
dtype: int64
```

Pandas Objects: DataFrame



- Pandas 주요 객체
 - Series: 인덱싱된 1차원 배열
 - DataFrame: 인덱싱된 2차원 배열($\approx$ 테이블)
 - 행 인덱스와 열 이름을 가짐
 - 인덱스를 공유하는 Series 객체들의 집합으로 볼 수 있음
 - 리스트/딕셔너리 또는 NumPy 배열로부터 생성 가능

Creating a DataFrame Object



- Series 객체 추가 생성

```
area_dict = {'California': 423967, 'Texas': 695662, 'New York': 141297,  
             'Florida': 170312, 'Illinois': 149995}
```

```
area = pd.Series(area_dict)
```

```
California    423967  
Texas         695662  
New York      141297  
Florida       170312  
Illinois      149995  
dtype: int64
```

Creating a DataFrame Object



- 두 Series 객체(population, area)를 포함하는 DataFrame 객체 생성

```
states = pd.DataFrame({'population': population, 'area': area})
```

	population	area
California	38332521	423967
Texas	26448193	695662
New York	19651127	141297
Florida	19552860	170312
Illinois	12882135	149995

Accessing DataFrame Objects

- Series 객체와 동일하게 index 속성을 통한 인덱스 레이블 접근

```
states.index
```

```
Index(['California', 'Texas', 'New York',  
      'Florida', 'Illinois'], dtype='object')
```

- column 속성: DataFrame 열 레이블 접근

```
states.columns
```

```
Index(['population', 'area'], dtype='object')
```


Accessing DataFrame Objects



- column 명을 이용한 Series 선택

```
states['area']
```

```
California    423967  
Texas         695662  
New York      141297  
Florida       170312  
Illinois      149995  
Name: area, dtype: int64
```

DataFrame 은 행/열 데이터 접근·조작이
용이한 NumPy 2차원 배열의 쉬운 버전

Creating a DataFrame Object from ndarray



- NumPy 2차원 배열로부터 생성

```
arr = np.random.rand(3, 2)
```

```
[[0.97800274 0.58767635]  
 [0.35633351 0.94177274]  
 [0.93060142 0.38300991]]
```

```
pd.DataFrame(arr, columns=['foo', 'bar'], index=['a', 'b', 'c'])
```

	foo	bar
a	0.978003	0.587676
b	0.356334	0.941773
c	0.930601	0.383010

Data Manipulation: Series



- Series 객체 조작: Dictionary 스타일

['a']	['b']	['c']	['d']
0.25	0.5	0.75	1.0

```
ser['b']    # indexing
```

```
0.5
```

```
'a' in ser  # check if key exists (dictionary-style)
```

```
True
```

```
0.25 in ser # check if value exists (not available!)
```

```
False
```

Data Manipulation: Series



['a']	['b']	['c']	['d']
0.25	0.5	0.75	1.0

```
ser.keys() # get the all keys(= indices)

Index(['a', 'b', 'c', 'd'], dtype='object')

ser['e'] = 1.25 # add a new value
ser['a'] = 0.125 # reassignment

a      0.125
b      0.500
c      0.750
d      1.000
e      1.250
dtype: float64
```

Data Manipulation: Series



- Series 객체 조작: 배열 스타일

- 슬라이싱

```
ser['a':'c']
```

['a']	['b']	['c']	['d']	['e']
0.125	0.5	0.75	1.0	1.25

- 마스크

```
ser[(ser > 0.3) & (ser < 0.8)]
```

['a']	['b']	['c']	['d']	['e']
0.125	0.5	0.75	1.0	1.25

- 팬시 인덱싱

```
ser[['a', 'e']]
```

['a']	['b']	['c']	['d']	['e']
0.125	0.5	0.75	1.0	1.25

Explicit / Implicit Indexing

- 그러나, 명시적/암묵적 인덱싱 방법에 따라 슬라이싱 결과가 다름

- 명시적 인덱싱^{explicit indexing}: 마지막 인덱스/값이 포함됨

```
ser['a':'c']
```

['a']	['b']	['c']	['d']	['e']
0.125	0.5	0.75	1.0	1.25

- 암묵적 인덱싱^{implicit indexing}: 마지막 인덱스/값이 포함되지 않음

```
ser[0:2]
```

[0]	[1]	[2]	[3]	[4]
0.125	0.5	0.75	1.0	1.25

Data Manipulation: DataFrame



- DataFrame 객체 조작
 - 특정 Series 객체 접근

```
states['area']
```

또는

```
states.area
```



```
California    423967
Texas         695662
New York      141297
Florida       170312
Illinois      149995
Name: area, dtype: int64
```

- `states[ 'column_name' ]` 방법이 활용성이 좋음
 - `states.state capital` (X)
 - `states['state capital']` (O)

Data Manipulation: DataFrame



- DataFrame 객체 조작
 - Series 객체 추가

```
states['density'] = states['population'] / states['area']
```

	population	area	density
California	38332521	423967	90.413926
Texas	26448193	695662	38.018740
New York	19651127	141297	139.076746
Florida	19552860	170312	114.806121
Illinois	12882135	149995	85.883763

Indexers

- **인덱서**: Pandas 객체의 명확한 인덱싱/슬라이싱을 지원하는 속성

– 주요 속성: `loc`, `iloc`

[1]	[3]	[5]	명시적 인덱스(레이블)
[0]	[1]	[2]	암묵적 인덱스(정수)
a	b	c	

- `loc` Label-based Locating: **명시적 인덱스를 참조하는 인덱싱/슬라이싱 수행**

`ser.loc[1]` → a

`ser.loc[1:3]` →

```

1    a
3    b
dtype: object

```

Indexers



['1'] ['3'] ['5']			명시적 인덱스(레이블)
[0] [1] [2]			암묵적 인덱스(정수)
a	b	c	

- `iloc` Integer-based Locating: **암묵적 인덱스**를 참조하는 인덱싱/슬라이싱 수행

`ser.iloc[1]` → `b`

`ser.iloc[1:3]` → `3      b`
`5      c`
`dtype: object`

Data Manipulation: DataFrame



- DataFrame = 2차원 배열
 - 배열 값들을 NumPy 배열로 추출, 예: `states.values`
 - 전치행렬 Tranposed Matrix 구하기, 예: `states.T`
 - 슬라이싱(암묵적 인덱싱), 예: `states.iloc[:3, :2]`
 - 슬라이싱(명시적 인덱싱), 예: `states.loc[: 'Illinois', : 'population']`

Data Manipulation: DataFrame



- `loc` 인덱서: **마스킹** + **팬시 인덱싱** 지원

```
states.loc[states.density > 100, ['population', 'density']]
```



	population	density
New York	19651127	139.076746
Florida	19552860	114.806121

- DataFrame 값 수정

```
states.iloc[0, 2] = 90
```



	population	area	density
California	19651127	423967	90.000000
Texas	19552860	695662	38.018740
New York	19651127	141297	139.076746
Florida	19552860	170312	114.806121
Illinois	12882135	149995	85.883763